

2)

K-fold

Cross Validation

Date _____

Page _____

Evaluating model performance

K-fold (cross) Validation

Why we need it:-

When we have many models (like logistic regression, SVM, Decision Tree, Random Forest), we want to know which model performs best.

Cross validation helps us measure model performance more accurately.

Δ Normal Train-Test Split

- We usually divide data into Train (eg. 70%) & Test (eg. 30%)

Example:-

A student practices 70 math questions (training) & is tested with 30 new ones (testing).

This checks how well he performs on unseen data.

Problem:-

If training & testing data has different types (eg. training = algebra, testing = calculus), the result becomes unreliable.

Imp

What is k-fold cross validation?

- Instead of doing one train-test split, we split the data into k equal parts (folds).
- Then we train and test multiple times, each time using a different fold for testing.

Example ($k=5$):

1. Divide 100 samples into 5 folds (20 samples each).

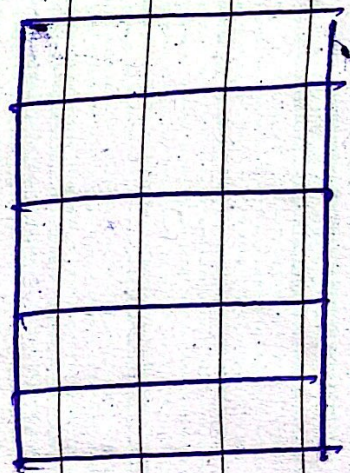
2. Run 5 iterations.

- Iteration 1: fold 1 = test, fold 2-5 = train
- Iteration 2: fold 2 = test, others = train
- ... Continue till fold 5 is iteration 5

3. Calculate accuracy for each iteration.

4. final score = Average of all 5 accuracies.

This gives a more reliable & fair evaluation since every data point is used once for testing.

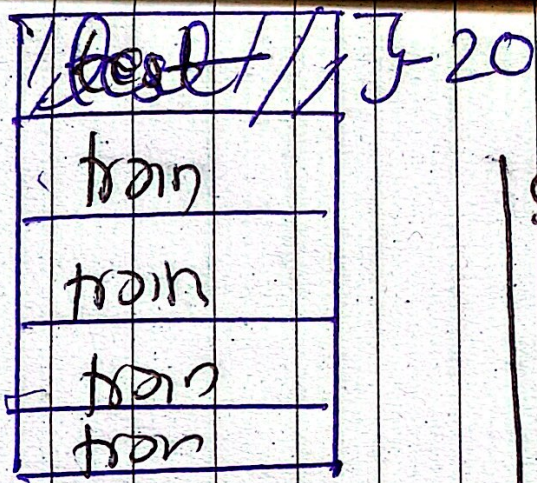


100 samples

(total dataset)



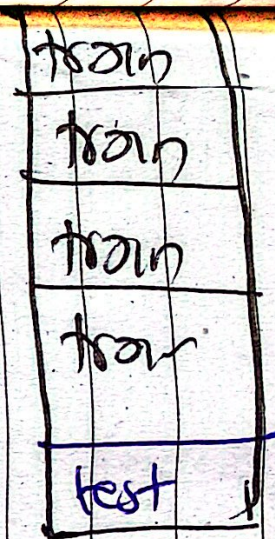
divided into K folds/parts



Score 1



Score 2



Scores

Average



✓ Average Score

Advantages of K-fold:-

uses all data efficiently (each sample used for both training & testing)

- Reduces bias due to random train-test split
- Helps compare multiple models fairly

Example Models Used:-

Dataset:- Digits dataset (handwritten numbers 0-9)

Tested Models:-

1. Logistic Regression
2. SVM
3. Random Forest

Observation:-

- Logistic Regression performed well
- SVM performed poorly
- Random Forest performed best (after tuning parameters like number of trees)

Problem with Normal Train-Test Split

- Each time you run it, accuracy changes slightly.

Example:-

First run = 95.9%

Second run = 95.3%

- Because data is randomly split each time.

That's why K-fold Cross Validation gives more stable results.

Stratified K-fold Cross Validation:-

- Similar to normal K-fold but preserves class proportion in each fold.

(eg. equal number of each flower type or digit in every fold)

- helps when dealing with classification problems

• get_score() function

A helper function written in code:

```
def get_score (model, x_train, x_test, y_train, y_test):
```

```
    model = fit(x_train, y_train)
```

```
    return model.score(x_test, y_test)
```

This trains the model & returns accuracy for given data.

• Cross-validation Using sklearn:-

Instead of writing loops manually, we can directly use:-

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score (model, x, y, cv=5)
```

Here,

model = classifier (eg. Logistic Regression (1))
X = features

y = target values

cv = 5 (Number of folds) $\rightarrow$ same as 2 split = 3 in k fold

This automatically performs k-fold cross validation & returns accuracies for each fold.

• Parameter Tuning Example:-

Try different parameters for same model:-

Random Forest Classifier (n_estimators = 5)

Random Forest Classifier (n_estimators = 40)

Random Forest Classifier (n_estimators = 50)

$\rightarrow$ As number of trees increases, accuracy improves (up to a point)

This process is called parameter tuning.

Summary -

Concept	Description
Train-test split	Divides data once (eg. 70-30). May give unstable result
K-fold	split data into k parts & tests k times. More accurate.
Stratified k-fold	ensures equal class distribution in folds
cross_val_score()	exists sklearn function for cross validation
Parameter Tuning	Try some model with different parameters to improve performance

Important

Key Idea:-

Machine learning model selection is not exact math - you must try different algorithms & different parameters using techniques like k-fold cross validation to find the best one.

(hyper parameters)

Python Code

Basic k-fold Example

from sklearn.model_selection import KFold

kf = KFold(n_splits=3)
 divide data into 3 equal parts (folds)
 for train_index, test_index in kf.split(
 [1, 2, 3, 4, 5, 6, 7, 8, 9]):
 print(train_index, test_index)

O/P

[3 4 5 6 7 8] [0 1 2] ✓
 [0 1 2 6 7 8] [3 4 5] ✓
 [0 1 2 3 4 5] [6 7 8] ✓

returns (index) (position)

that we can use to pick training & testing example

first fold → 0-2 for testing, rest training
 second fold → 3-5 for testing, rest training
 third fold → 6-8 for testing, rest training

kf = split(X, Y)
 input data points (compulsory)
 optional (target)

Date _____
Page _____

K-Means

PCA

13. Kmeans Clustering Algorithm

Logic
Step 1:- $n \text{ split} = 3$
means that we divide the data into 3 folds.

Our data = [1, 2, 3, 4, 5, 6, 7, 8, 9] $\rightarrow$ total 9 samples

Now k fold divides them evenly through (index)

Fold 1 $\rightarrow$ [0, 1, 2]
Fold 2 $\rightarrow$ [3, 4, 5]
Fold 3 $\rightarrow$ [6, 7, 8]

Each fold = 3 samples
 $\because (9 \div 3 = 3)$

Step 2:- Kfold works in Rounds

There are 3 rounds (iterations) because we have 3 folds.

- In each round, 1 fold is used for testing.
- and the other 2 folds (remaining) are used for training.

Step 3:- What Happens in Each Round

Round	Training Indexes (Rest training)	Testing Indexes $\rightarrow$ 3 samples
1	[3, 4, 5, 6, 7, 8] $\rightarrow$ 6 samples	[0, 1, 2] $\rightarrow$ 3 samples
2	[0, 1, 2, 6, 7, 8] $\rightarrow$ 6 samples	[3, 4, 5] $\rightarrow$ 3 samples
3	[0, 1, 2, 3, 4, 5] $\rightarrow$ 6 samples	[6, 7, 8] $\rightarrow$ 3 samples

So, We can always see 6 samples for training & 3 samples for testing.

"If you make K folds, each round use $(K-1)$ folds for training & 1 fold for testing".
for K times

So, for $K=3 \rightarrow$ training = 2 folds $\rightarrow$ 6 samples
testing = 1 fold $\rightarrow$ 3 samples.

for $K=5 \rightarrow$ training = 4 folds $\rightarrow$ 80% samples

for $K=10 \rightarrow$ training = 9 folds $\rightarrow$ 90% samples.

★ Real K-fold Example

try Basic k-fold example then this

def get_score(model, X_train, X_test, y_train, y_test):

model.fit(X_train, y_train)
return model.score(X_test, y_test)

from sklearn.model_selection import Stratified KFold

fold = StratifiedKFold(n_splits=3)

scores_l = [] → for score of linear regression

scores_svm = [] → for score of SVM

scores_rf = [] → for score of random forest

for train_index, test_index in fold.split(
digits.data, digits.target):

Directly

X_train, X_test = digits.data[train_index],
digits.data[test_index]

y_train, y_test = digits.target[train_index],
digits.target[test_index]

scores_l.append(get_score(LogisticRegression(), X_train,
X_test, y_train, y_test))

scores_svm.append(get_score(SVC(), X_train, X_test,
y_train, y_test))

scores_rf.append(get_score(RandomForestClassifier(),
X_train, X_test, y_train, y_test))

But Directly, we could we

Sklearn's cross validation (without
writing any of that code)

from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=3)
scores or
scores.mean()

or
np.average(scores)

Eg -

scores = cross_val_score(model, digits.data, digits.target,
cv=3)

not only
X_test, y_test